

# Algorithmique

## Réversibilité

**Cédric Lhoussaine**

Université de Lille / IUT de Lille

## » Outline

### Récurtivité

### Tri par fusion

### Sous-tableau maximal

### Tri rapide

## » Réversibilité

### Réversibilité

Un fonction est dite **réursive** si s'appellent elle-même

## » Récursivité

### Récursivité

Un fonction est dite **récursive** si s'appellent elle-même

- \* Permet de ramener la résolution d'un problème compliqué à la résolution d'un (ou plusieurs) problèmes plus simples
- \* Principe de la récursivité :
  - \* Il doit toujours y avoir (au moins) un cas de terminaison (i.e. sans appel récursif)
  - \* Chaque appel récursif doit garantir qu'on se « rapproche » (strictement) un peu plus d'un cas de terminaison

## » Récurtivité

- \* **Avantages**

- \* concision
- \* clarté
- \* preuve de correction et calculs de complexité facilités

## » Récursivité

### \* Avantages

- \* concision
- \* clarté
- \* preuve de correction et calculs de complexité facilités

### \* Inconvénients

- \* requiert en général plus d'espace mémoire (croissance de la pile d'exécution)
- \* mais, dans certains cas (récursivité terminale), le compilateur (mais pas celui de java!) peut optimiser les programmes récursifs en les transformant automatiquement en programmes itératifs

## » Diviser pour régner

Paradigme récursif qui consiste en 3 étapes :

- \* **Diviser** le problème en sous-problèmes
- \* **Résoudre** chacun des sous-problèmes de façon récursive sauf ceux d'une taille suffisamment petite pour être résolus directement
- \* **Combiner** les solutions aux sous-problèmes pour obtenir une solution au problème d'origine

## » Tri par fusion

- \* Principe général (suivant le paradigme *diviser pour régner*)
  - \* **Diviser** la suite des  $n$  éléments à trier en 2 sous-suites de longueur  $n/2$
  - \* **Résoudre récursivement** le tri de chacune des 2 sous-suites si leur longueur est  $> 2$
  - \* **Combiner** les sous-suites triées pour obtenir la suite complète triée : c'est la **fusion**
- \* Comment fusionner 2 listes triées  $\ell_1$  et  $\ell_2$  ?
  - \* On traite  $\ell_1$  et  $\ell_2$  comme des **files**



## » Tri par fusion

- \* Principe général (suivant le paradigme *diviser pour régner*)
  - \* **Diviser** la suite des  $n$  éléments à trier en 2 sous-suites de longueur  $n/2$
  - \* **Résoudre récursivement** le tri de chacune des 2 sous-suites si leur longueur est  $> 2$
  - \* **Combiner** les sous-suites triées pour obtenir la suite complète triée : c'est la **fusion**
- \* Comment fusionner 2 listes triées  $\ell_1$  et  $\ell_2$  ?
  - \* On traite  $\ell_1$  et  $\ell_2$  comme des **pires**
  - \*  $\ell_1 = \langle 2, 5, 7 \rangle$ ,  $\ell_2 = \langle 3, 8, 10 \rangle$ , sortie =  $\langle \rangle$

## » Tri par fusion

- \* Principe général (suivant le paradigme *diviser pour régner*)
  - \* **Diviser** la suite des  $n$  éléments à trier en 2 sous-suites de longueur  $n/2$
  - \* **Résoudre récursivement** le tri de chacune des 2 sous-suites si leur longueur est  $> 2$
  - \* **Combiner** les sous-suites triées pour obtenir la suite complète triée : c'est la **fusion**
- \* Comment fusionner 2 listes triées  $\ell_1$  et  $\ell_2$ ?
  - \* On traite  $\ell_1$  et  $\ell_2$  comme des **pires**
  - \*  $\ell_1 = \langle 2, 5, 7 \rangle$ ,  $\ell_2 = \langle 3, 8, 10 \rangle$ , sortie =  $\langle \rangle$  (2<3)

## » Tri par fusion

- \* Principe général (suivant le paradigme *diviser pour régner*)
  - \* **Diviser** la suite des  $n$  éléments à trier en 2 sous-suites de longueur  $n/2$
  - \* **Résoudre récursivement** le tri de chacune des 2 sous-suites si leur longueur est  $> 2$
  - \* **Combiner** les sous-suites triées pour obtenir la suite complète triée : c'est la **fusion**
- \* Comment fusionner 2 listes triées  $\ell_1$  et  $\ell_2$ ?
  - \* On traite  $\ell_1$  et  $\ell_2$  comme des **pires**
  - \*  $\ell_1 = \langle 2, 5, 7 \rangle$ ,  $\ell_2 = \langle 3, 8, 10 \rangle$ , sortie =  $\langle \rangle$  (2<3)
  - \*  $\ell_1 = \langle 5, 7 \rangle$ ,  $\ell_2 = \langle 3, 8, 10 \rangle$ , sortie =  $\langle 2 \rangle$

## » Tri par fusion

- \* Principe général (suivant le paradigme *diviser pour régner*)
  - \* **Diviser** la suite des  $n$  éléments à trier en 2 sous-suites de longueur  $n/2$
  - \* **Résoudre récursivement** le tri de chacune des 2 sous-suites si leur longueur est  $> 2$
  - \* **Combiner** les sous-suites triées pour obtenir la suite complète triée : c'est la **fusion**
- \* Comment fusionner 2 listes triées  $\ell_1$  et  $\ell_2$ ?
  - \* On traite  $\ell_1$  et  $\ell_2$  comme des **pires**
  - \*  $\ell_1 = \langle 2, 5, 7 \rangle$ ,  $\ell_2 = \langle 3, 8, 10 \rangle$ , sortie =  $\langle \rangle$  (2<3)
  - \*  $\ell_1 = \langle 5, 7 \rangle$ ,  $\ell_2 = \langle 3, 8, 10 \rangle$ , sortie =  $\langle 2 \rangle$  (3<5)

## » Tri par fusion

- \* Principe général (suivant le paradigme *diviser pour régner*)
  - \* **Diviser** la suite des  $n$  éléments à trier en 2 sous-suites de longueur  $n/2$
  - \* **Résoudre récursivement** le tri de chacune des 2 sous-suites si leur longueur est  $> 2$
  - \* **Combiner** les sous-suites triées pour obtenir la suite complète triée : c'est la **fusion**
- \* Comment fusionner 2 listes triées  $\ell_1$  et  $\ell_2$  ?
  - \* On traite  $\ell_1$  et  $\ell_2$  comme des **pires**
  - \*  $\ell_1 = \langle 2, 5, 7 \rangle$ ,  $\ell_2 = \langle 3, 8, 10 \rangle$ , sortie =  $\langle \rangle$  (2<3)
  - \*  $\ell_1 = \langle 5, 7 \rangle$ ,  $\ell_2 = \langle 3, 8, 10 \rangle$ , sortie =  $\langle 2 \rangle$  (3<5)
  - \*  $\ell_1 = \langle 5, 7 \rangle$ ,  $\ell_2 = \langle 8, 10 \rangle$ , sortie =  $\langle 2, 3 \rangle$

## » Tri par fusion

- \* Principe général (suivant le paradigme *diviser pour régner*)
  - \* **Diviser** la suite des  $n$  éléments à trier en 2 sous-suites de longueur  $n/2$
  - \* **Résoudre récursivement** le tri de chacune des 2 sous-suites si leur longueur est  $> 2$
  - \* **Combiner** les sous-suites triées pour obtenir la suite complète triée : c'est la **fusion**
- \* Comment fusionner 2 listes triées  $\ell_1$  et  $\ell_2$ ?
  - \* On traite  $\ell_1$  et  $\ell_2$  comme des **pires**
  - \*  $\ell_1 = \langle 2, 5, 7 \rangle$ ,  $\ell_2 = \langle 3, 8, 10 \rangle$ , sortie =  $\langle \rangle$  (2<3)
  - \*  $\ell_1 = \langle 5, 7 \rangle$ ,  $\ell_2 = \langle 3, 8, 10 \rangle$ , sortie =  $\langle 2 \rangle$  (3<5)
  - \*  $\ell_1 = \langle 5, 7 \rangle$ ,  $\ell_2 = \langle 8, 10 \rangle$ , sortie =  $\langle 2, 3 \rangle$  (5<8)

## » Tri par fusion

- \* Principe général (suivant le paradigme *diviser pour régner*)
  - \* **Diviser** la suite des  $n$  éléments à trier en 2 sous-suites de longueur  $n/2$
  - \* **Résoudre récursivement** le tri de chacune des 2 sous-suites si leur longueur est  $> 2$
  - \* **Combiner** les sous-suites triées pour obtenir la suite complète triée : c'est la **fusion**
- \* Comment fusionner 2 listes triées  $\ell_1$  et  $\ell_2$ ?
  - \* On traite  $\ell_1$  et  $\ell_2$  comme des **pires**
  - \*  $\ell_1 = \langle 2, 5, 7 \rangle$ ,  $\ell_2 = \langle 3, 8, 10 \rangle$ , sortie =  $\langle \rangle$  (2<3)
  - \*  $\ell_1 = \langle 5, 7 \rangle$ ,  $\ell_2 = \langle 3, 8, 10 \rangle$ , sortie =  $\langle 2 \rangle$  (3<5)
  - \*  $\ell_1 = \langle 5, 7 \rangle$ ,  $\ell_2 = \langle 8, 10 \rangle$ , sortie =  $\langle 2, 3 \rangle$  (5<8)
  - \*  $\ell_1 = \langle 7 \rangle$ ,  $\ell_2 = \langle 8, 10 \rangle$ , sortie =  $\langle 2, 3, 5 \rangle$

## » Tri par fusion

- \* Principe général (suivant le paradigme *diviser pour régner*)
  - \* **Diviser** la suite des  $n$  éléments à trier en 2 sous-suites de longueur  $n/2$
  - \* **Résoudre récursivement** le tri de chacune des 2 sous-suites si leur longueur est  $> 2$
  - \* **Combiner** les sous-suites triées pour obtenir la suite complète triée : c'est la **fusion**
- \* Comment fusionner 2 listes triées  $\ell_1$  et  $\ell_2$  ?
  - \* On traite  $\ell_1$  et  $\ell_2$  comme des **pires**
  - \*  $\ell_1 = \langle 2, 5, 7 \rangle$ ,  $\ell_2 = \langle 3, 8, 10 \rangle$ , sortie =  $\langle \rangle$  (2<3)
  - \*  $\ell_1 = \langle 5, 7 \rangle$ ,  $\ell_2 = \langle 3, 8, 10 \rangle$ , sortie =  $\langle 2 \rangle$  (3<5)
  - \*  $\ell_1 = \langle 5, 7 \rangle$ ,  $\ell_2 = \langle 8, 10 \rangle$ , sortie =  $\langle 2, 3 \rangle$  (5<8)
  - \*  $\ell_1 = \langle 7 \rangle$ ,  $\ell_2 = \langle 8, 10 \rangle$ , sortie =  $\langle 2, 3, 5 \rangle$  (7<8)



## » Tri par fusion

- \* Principe général (suivant le paradigme *diviser pour régner*)
  - \* **Diviser** la suite des  $n$  éléments à trier en 2 sous-suites de longueur  $n/2$
  - \* **Résoudre récursivement** le tri de chacune des 2 sous-suites si leur longueur est  $> 2$
  - \* **Combiner** les sous-suites triées pour obtenir la suite complète triée : c'est la **fusion**
- \* Comment fusionner 2 listes triées  $\ell_1$  et  $\ell_2$  ?
  - \* On traite  $\ell_1$  et  $\ell_2$  comme des **pires**
  - \*  $\ell_1 = \langle 2, 5, 7 \rangle$ ,  $\ell_2 = \langle 3, 8, 10 \rangle$ , sortie =  $\langle \rangle$  (2<3)
  - \*  $\ell_1 = \langle 5, 7 \rangle$ ,  $\ell_2 = \langle 3, 8, 10 \rangle$ , sortie =  $\langle 2 \rangle$  (3<5)
  - \*  $\ell_1 = \langle 5, 7 \rangle$ ,  $\ell_2 = \langle 8, 10 \rangle$ , sortie =  $\langle 2, 3 \rangle$  (5<8)
  - \*  $\ell_1 = \langle 7 \rangle$ ,  $\ell_2 = \langle 8, 10 \rangle$ , sortie =  $\langle 2, 3, 5 \rangle$  (7<8)
  - \*  $\ell_1 = \langle \rangle$ ,  $\ell_2 = \langle 8, 10 \rangle$ , sortie =  $\langle 2, 3, 5, 7 \rangle$

## » Tri par fusion

- \* Principe général (suivant le paradigme *diviser pour régner*)
  - \* **Diviser** la suite des  $n$  éléments à trier en 2 sous-suites de longueur  $n/2$
  - \* **Résoudre récursivement** le tri de chacune des 2 sous-suites si leur longueur est  $> 2$
  - \* **Combiner** les sous-suites triées pour obtenir la suite complète triée : c'est la **fusion**
- \* Comment fusionner 2 listes triées  $\ell_1$  et  $\ell_2$  ?
  - \* On traite  $\ell_1$  et  $\ell_2$  comme des **pires**
  - \*  $\ell_1 = \langle 2, 5, 7 \rangle$ ,  $\ell_2 = \langle 3, 8, 10 \rangle$ , sortie =  $\langle \rangle$  (2<3)
  - \*  $\ell_1 = \langle 5, 7 \rangle$ ,  $\ell_2 = \langle 3, 8, 10 \rangle$ , sortie =  $\langle 2 \rangle$  (3<5)
  - \*  $\ell_1 = \langle 5, 7 \rangle$ ,  $\ell_2 = \langle 8, 10 \rangle$ , sortie =  $\langle 2, 3 \rangle$  (5<8)
  - \*  $\ell_1 = \langle 7 \rangle$ ,  $\ell_2 = \langle 8, 10 \rangle$ , sortie =  $\langle 2, 3, 5 \rangle$  (7<8)
  - \*  $\ell_1 = \langle \rangle$ ,  $\ell_2 = \langle 8, 10 \rangle$ , sortie =  $\langle 2, 3, 5, 7 \rangle$
  - \*  $\ell_1 = \langle \rangle$ ,  $\ell_2 = \langle \rangle$ , **sortie** =  $\langle 2, 3, 5, 7, 8, 10 \rangle$

## » Fusion

Pour simplifier (en fait, supprimer) la dernière étape, on ajoute une valeur  $\infty$  à  $\ell_1$  et  $\ell_2$

$$* \ell_1 = \langle 2, 5, 7, \boxed{\infty} \rangle, \ell_2 = \langle 3, 8, 10, \boxed{\infty} \rangle, \text{sortie} = \langle \rangle$$

## » Fusion

Pour simplifier (en fait, supprimer) la dernière étape, on ajoute une valeur  $\infty$  à  $\ell_1$  et  $\ell_2$

$$* \ell_1 = \langle 2, 5, 7, \boxed{\infty} \rangle, \ell_2 = \langle 3, 8, 10, \boxed{\infty} \rangle, \text{sortie} = \langle \rangle$$

\* ...

$$* \ell_1 = \langle \infty \rangle, \ell_2 = \langle 8, 10, \infty \rangle, \text{sortie} = \langle 2, 3, 5, 7 \rangle \quad (8 < \infty)$$

## » Fusion

Pour simplifier (en fait, supprimer) la dernière étape, on ajoute une valeur  $\infty$  à  $\ell_1$  et  $\ell_2$

$$* \ell_1 = \langle 2, 5, 7, \boxed{\infty} \rangle, \ell_2 = \langle 3, 8, 10, \boxed{\infty} \rangle, \text{sortie} = \langle \rangle$$

\* ...

$$* \ell_1 = \langle \infty \rangle, \ell_2 = \langle 8, 10, \infty \rangle, \text{sortie} = \langle 2, 3, 5, 7 \rangle \quad (8 < \infty)$$

$$* \ell_1 = \langle \infty \rangle, \ell_2 = \langle 10, \infty \rangle, \text{sortie} = \langle 2, 3, 5, 7, 8 \rangle \quad (10 < \infty)$$

## » Fusion

Pour simplifier (en fait, supprimer) la dernière étape, on ajoute une valeur  $\infty$  à  $\ell_1$  et  $\ell_2$

$$* \ell_1 = \langle 2, 5, 7, \boxed{\infty} \rangle, \ell_2 = \langle 3, 8, 10, \boxed{\infty} \rangle, \text{sortie} = \langle \rangle$$

\* ...

$$* \ell_1 = \langle \infty \rangle, \ell_2 = \langle 8, 10, \infty \rangle, \text{sortie} = \langle 2, 3, 5, 7 \rangle \quad (8 < \infty)$$

$$* \ell_1 = \langle \infty \rangle, \ell_2 = \langle 10, \infty \rangle, \text{sortie} = \langle 2, 3, 5, 7, 8 \rangle \quad (10 < \infty)$$

$$* \ell_1 = \langle \infty \rangle, \ell_2 = \langle \infty \rangle, \text{sortie} = \langle 2, 3, 5, 7, 8, 10 \rangle$$

## » Fusion

Algorithme de fusion des sous-tableaux  $A[p \dots q]$  et  $A[q + 1, \dots, r]$

```
FUSION(A, p, q, r)
  n1 = q - p + 1
  n2 = r - q
  nouveaux tableaux L[0, ..., n1] et R[0, ..., n2]
  for i = 0 to n1 - 1 do
    L[i] = A[p + i]
  for j = 0 to n2 - 1 do
    R[j] = A[q + j + 1]
  L[n1] = ∞
  R[n2] = ∞
  i = j = 0
  for k = p to r do
    if L[i] ≤ R[j] then
      A[k] = L[i]
      i = i + 1
    else
      A[k] = R[j]
      j = j + 1
```

## » Correction de l'algorithme de fusion

```
FUSION(A, p, q, r)
  n1 = q - p + 1
  n2 = r - q
  nouveaux tableaux L[0...n1] et R[0...n2]
  for i = 0 to n1-1 do
    L[i] = A[p+i]
  for j = 0 to n2-1 do
    R[j] = A[q+j+1]
  L[n1+1] = ∞
  R[n2+1] = ∞
  i = j = 1
  for k = p to r do
    if L[i] <= R[j] then
      A[k] = L[i]
      i = i+1
    else
      A[k] = R[j]
      j = j+1
```

$Inv(k) =$

- \*  $A[p..k-1]$  contient les  $k-p$  plus petits éléments de  $L[0 \dots n_1]$  et  $R[0 \dots n_2]$  en ordre trié, et
- \*  $L[i]$  et  $R[j]$  sont les plus petits éléments à ne pas avoir été copiés dans  $A$ .



## » Complexité de l'algorithme de fusion

```
FUSION(A,p,q,r)
  n1 = q-p+1
  n2 = r-q
  nouveaux tableaux L[0,..,n1] et R[0..n2]
  for i = 0 to n1-1 do
    L[i] = A[p+i]
  for j = 0 to n2-1 do
    R[j] = A[q+j+1]
  L[n1+1] = ∞
  R[n2+1] = ∞
  i = j = 1
  for k = p to r do
    if L[i] <= R[j] then
      A[k] = L[i]
      i = i+1
    else
      A[k] = R[j]
      j = j+1
```

Complexité de la fusion de  $n$  éléments ?

## » Complexité de l'algorithme de fusion

```
FUSION(A,p,q,r)
  n1 = q-p+1
  n2 = r-q
  nouveaux tableaux L[0,..,n1] et R[0..n2]
  for i = 0 to n1-1 do
    L[i] = A[p+i]
  for j = 0 to n2-1 do
    R[j] = A[q+j+1]
  L[n1+1] = ∞
  R[n2+1] = ∞
  i = j = 1
  for k = p to r do
    if L[i] <= R[j] then
      A[k] = L[i]
      i = i+1
    else
      A[k] = R[j]
      j = j+1
```

Complexité de la fusion de  $n$  éléments ?

$$\Theta(n)$$

## » Tri par fusion

Algorithme de tri récursif :

```
TRI-FUSION(A, p, r)
  if p < r then
    q = inf((p+r)/2)
    TRI-FUSION(A, p, q)
    TRI-FUSION(A, q+1, r)
    FUSION(A, p, q, r)
```

## » Correction du tri par fusion

Preuve par récurrence sur la taille  $n$  de  $A$  à trier.

\* Si  $n = 1$ ,

```
TRI-FUSION(A, p, r)
  if p < r then
    q = inf((p+r)/2)
    TRI-FUSION(A, p, q)
    TRI-FUSION(A, q+1, r)
    FUSION(A, p, q, r)
```

## » Correction du tri par fusion

Preuve par récurrence sur la taille  $n$  de  $A$  à trier.

- \* Si  $n = 1$ ,  $A$  est déjà trié et TRI-FUSION ne fait rien, ce qui est le comportement attendu

```
TRI-FUSION(A, p, r)
  if p < r then
    q = inf((p+r)/2)
    TRI-FUSION(A, p, q)
    TRI-FUSION(A, q+1, r)
    FUSION(A, p, q, r)
```

## » Correction du tri par fusion

```
TRI-FUSION(A, p, r)
  if p < r then
    q = inf((p+r)/2)
    TRI-FUSION(A, p, q)
    TRI-FUSION(A, q+1, r)
    FUSION(A, p, q, r)
```

Preuve par récurrence sur la taille  $n$  de  $A$  à trier.

- \* Si  $n = 1$ ,  $A$  est déjà trié et TRI-FUSION ne fait rien, ce qui est le comportement attendu
- \* (HI) Supposons que TRI-FUSION est correct pour tout tableau de taille  $n \leq k$  pour un certain  $k \geq 1$
- \* Si  $n = k + 1$ ,

## » Correction du tri par fusion

```
TRI-FUSION(A, p, r)
  if p < r then
    q = inf((p+r)/2)
    TRI-FUSION(A, p, q)
    TRI-FUSION(A, q+1, r)
    FUSION(A, p, q, r)
```

Preuve par récurrence sur la taille  $n$  de  $A$  à trier.

- \* Si  $n = 1$ ,  $A$  est déjà trié et TRI-FUSION ne fait rien, ce qui est le comportement attendu
- \* (HI) Supposons que TRI-FUSION est correct pour tout tableau de taille  $n \leq k$  pour un certain  $k \geq 1$
- \* Si  $n = k + 1$ , TRI-FUSION est appelé sur 2 tableaux de tailles  $n_1$  et  $n_2$  de taille  $\leq k$ . Il en résulte, par HI, que  $A[p \dots q]$  et  $A[q + 1 \dots r]$  sont triés. Par correction de l'algorithme FUSION,  $A[p \dots r]$  est trié à l'issue de TRI-FUSION.

## » Complexité du tri par fusion

On suppose, pour simplifier, que  $n$  est une puissance de 2.

Soit  $T(n)$  le temps d'exécution dans le cas le plus défavorable de TRI-FUSION sur un tableau de taille  $n$ .



## » Complexité du tri par fusion

On suppose, pour simplifier, que  $n$  est une puissance de 2.

Soit  $T(n)$  le temps d'exécution dans le cas le plus défavorable de TRI-FUSION sur un tableau de taille  $n$ .

\*  $T(1) = \Theta(1)$  (temps constant pour un tableau de taille 1)

## » Complexité du tri par fusion

On suppose, pour simplifier, que  $n$  est une puissance de 2.

Soit  $T(n)$  le temps d'exécution dans le cas le plus défavorable de TRI-FUSION sur un tableau de taille  $n$ .

- \*  $T(1) = \Theta(1)$  (temps constant pour un tableau de taille 1)
- \* si  $n > 1$ 
  - \* **Diviser** (calcul de  $\lfloor (p+r)/2 \rfloor$ ) =  $\Theta(1)$

## » Complexité du tri par fusion

On suppose, pour simplifier, que  $n$  est une puissance de 2.

Soit  $T(n)$  le temps d'exécution dans le cas le plus défavorable de TRI-FUSION sur un tableau de taille  $n$ .

- \*  $T(1) = \Theta(1)$  (temps constant pour un tableau de taille 1)
- \* si  $n > 1$ 
  - \* **Diviser** (calcul de  $\lfloor (p + r)/2 \rfloor$ ) =  $\Theta(1)$
  - \* **Résolution** (2 appels récurifs sur des tableaux de taille  $n/2$ ) =

## » Complexité du tri par fusion

On suppose, pour simplifier, que  $n$  est une puissance de 2.

Soit  $T(n)$  le temps d'exécution dans le cas le plus défavorable de TRI-FUSION sur un tableau de taille  $n$ .

- \*  $T(1) = \Theta(1)$  (temps constant pour un tableau de taille 1)
- \* si  $n > 1$ 
  - \* **Diviser** (calcul de  $\lfloor (p+r)/2 \rfloor$ ) =  $\Theta(1)$
  - \* **Résolution** (2 appels récurifs sur des tableaux de taille  $n/2$ ) =  $2 * T(n/2)$
  - \* **Combiner** (appel à FUSION sur un tableau de taille  $n$ ) =  $\Theta(n)$

Equation de récurrence de  $T(n)$

$$T(n) = \begin{cases} \Theta(1) & \text{si } n = 1; \\ 2T(n/2) + \Theta(n) & \text{si } n > 1 \end{cases}$$

## » Complexité du tri par fusion

On suppose, pour simplifier, que  $n$  est une puissance de 2.

Soit  $T(n)$  le temps d'exécution dans le cas le plus défavorable de TRI-FUSION sur un tableau de taille  $n$ .

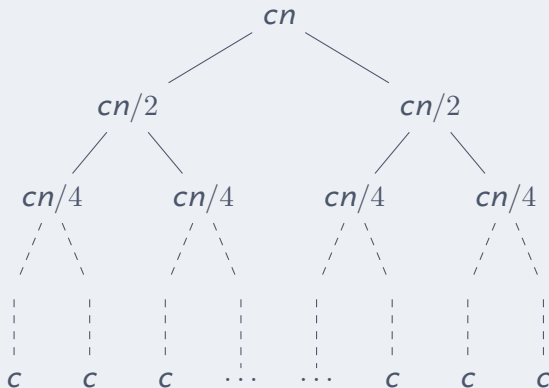
- \*  $T(1) = \Theta(1)$  (temps constant pour un tableau de taille 1)
- \* si  $n > 1$ 
  - \* **Diviser** (calcul de  $\lfloor (p+r)/2 \rfloor$ ) =  $\Theta(1)$
  - \* **Résolution** (2 appels récurifs sur des tableaux de taille  $n/2$ ) =  $2 * T(n/2)$
  - \* **Combiner** (appel à FUSION sur un tableau de taille  $n$ ) =  $\Theta(n)$

Equation de récurrence de  $T(n)$

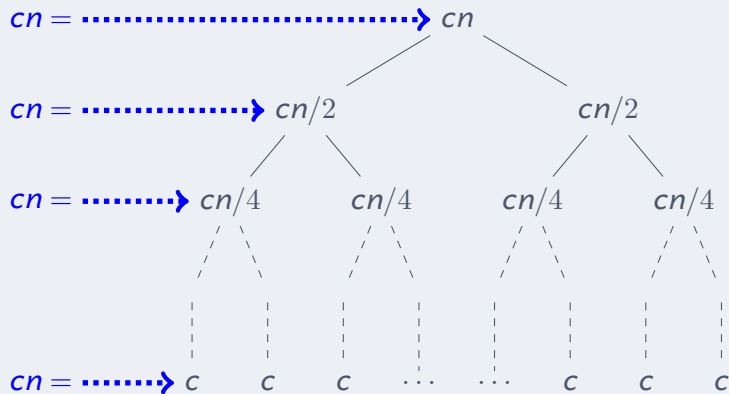
$$T(n) = \begin{cases} \Theta(1) & \text{si } n = 1; \\ 2T(n/2) + \Theta(n) & \text{si } n > 1 \end{cases}$$

Solution ?...

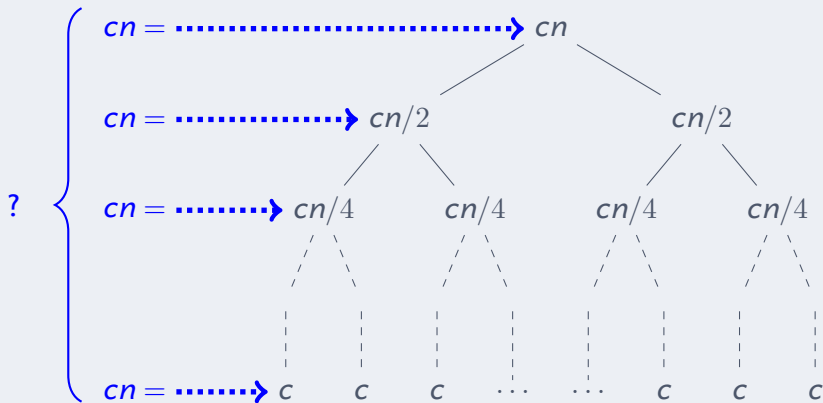
## » Complexité du tri par fusion



## » Complexité du tri par fusion

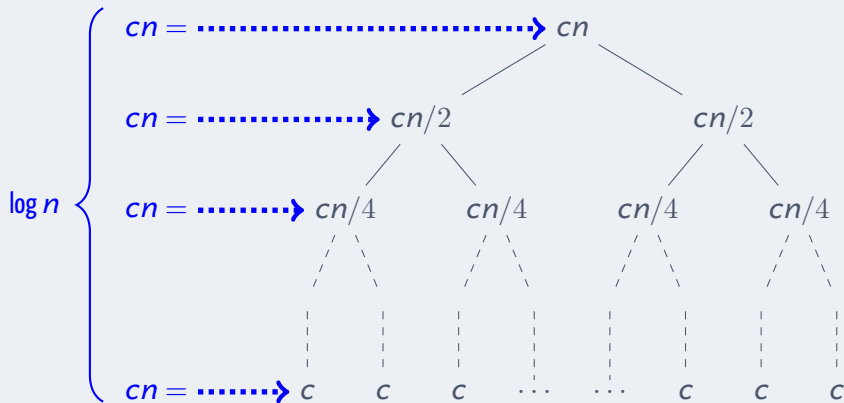


## » Complexité du tri par fusion

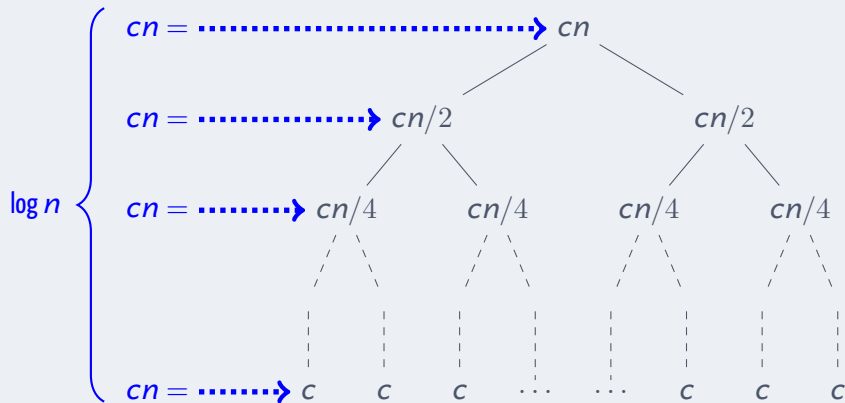




## » Complexité du tri par fusion



## » Complexité du tri par fusion



$$\Rightarrow T(n) = \Theta(n \log n)$$

## » Le problème du sous-tableau maximum

Soit une suite (connue) de valeurs d'un produit pendant  $n$  jours consécutifs, on souhaite déterminer quand acheter puis vendre le produit pour maximiser son profit.

## » Le problème du sous-tableau maximum

Soit une suite (connue) de valeurs d'un produit pendant  $n$  jours consécutifs, on souhaite déterminer quand acheter puis vendre le produit pour maximiser son profit.

\* Exemple :

| Jours | 0   | 1   | 2   | 3  | 4   | 5   | 6  | 7  |
|-------|-----|-----|-----|----|-----|-----|----|----|
| Cours | 100 | 113 | 110 | 85 | 102 | 105 | 63 | 70 |

- \* Achat jour 0, vente jour 2  $\Rightarrow$  +10
- \* Achat jour 3, vente jour 5  $\Rightarrow$  +20

## » Une première proposition d'algorithme

Solution par « force brute » (*brute force*) : énumérer toutes les paires de dates pour lesquelles la date d'achat précède la date de vente.

Combien y a-t-il de telles paires ?

## » Une première proposition d'algorithme

Solution par « force brute » (*brute force*) : énumérer toutes les paires de dates pour lesquelles la date d'achat précède la date de vente.

Combien y a-t-il de telles paires ?

$$\binom{n}{2} = \Theta(n^2) \text{ dans le pire des cas}$$

## » Une première proposition d'algorithme

Solution par « force brute » (*brute force*) : énumérer toutes les paires de dates pour lesquelles la date d'achat précède la date de vente.

Combien y a-t-il de telles paires ?

$$\binom{n}{2} = \Theta(n^2) \text{ dans le pire des cas}$$

Peut-on faire mieux ?...

## » Une deuxième proposition d'algorithme

Soit acheter au prix minimal soit vendre au jour maximal

- \* idée de l'algorithme :
  - \* trouver le cours maximal ( $M$ ) et minimal ( $m$ )



## » Une deuxième proposition d'algorithme

Soit acheter au prix minimal soit vendre au jour maximal

- \* idée de l'algorithme :
  - \* trouver le cours maximal ( $M$ ) et minimal ( $m$ )
  - \* chercher le cours minimal ( $m'$ ) antérieur au cours maximal  $M$

| Jours | 0 | 1    | 2   | 3 | 4   | 5 | 6   | 7 |
|-------|---|------|-----|---|-----|---|-----|---|
| Cours |   | $m'$ | ... |   | $M$ |   | ... |   |

## » Une deuxième proposition d'algorithme

Soit acheter au prix minimal soit vendre au jour maximal

\* idée de l'algorithme :

\* trouver le cours maximal ( $M$ ) et minimal ( $m$ )

\* chercher le cours minimal ( $m'$ ) antérieur au cours maximal  $M$

|       |   |      |     |   |     |   |     |   |
|-------|---|------|-----|---|-----|---|-----|---|
| Jours | 0 | 1    | 2   | 3 | 4   | 5 | 6   | 7 |
| Cours |   | $m'$ | ... |   | $M$ |   | ... |   |

\* chercher le cours maximal ( $M'$ ) postérieur au cours minimal  $m$

|       |   |   |     |   |   |     |     |      |
|-------|---|---|-----|---|---|-----|-----|------|
| Jours | 0 | 1 | 2   | 3 | 4 | 5   | 6   | 7    |
| Cours |   |   | ... |   |   | $m$ | ... | $M'$ |

## » Une deuxième proposition d'algorithme

Soit acheter au prix minimal soit vendre au jour maximal

\* idée de l'algorithme :

\* trouver le cours maximal ( $M$ ) et minimal ( $m$ )

\* chercher le cours minimal ( $m'$ ) antérieur au cours maximal  $M$

|       |   |      |     |   |     |   |     |   |
|-------|---|------|-----|---|-----|---|-----|---|
| Jours | 0 | 1    | 2   | 3 | 4   | 5 | 6   | 7 |
| Cours |   | $m'$ | ... |   | $M$ |   | ... |   |

\* chercher le cours maximal ( $M'$ ) postérieur au cours minimal  $m$

|       |   |   |     |   |   |     |     |      |
|-------|---|---|-----|---|---|-----|-----|------|
| Jours | 0 | 1 | 2   | 3 | 4 | 5   | 6   | 7    |
| Cours |   |   | ... |   |   | $m$ | ... | $M'$ |

\* renvoyer la meilleure paire ( $M, m'$ ) ou ( $M', m$ )

## » Une deuxième proposition d'algorithme

Soit acheter au prix minimal soit vendre au jour maximal

- \* idée de l'algorithme :

- \* trouver le cours maximal ( $M$ ) et minimal ( $m$ )

- \* chercher le cours minimal ( $m'$ ) antérieur au cours maximal  $M$

|       |   |      |     |   |     |   |     |   |
|-------|---|------|-----|---|-----|---|-----|---|
| Jours | 0 | 1    | 2   | 3 | 4   | 5 | 6   | 7 |
| Cours |   | $m'$ | ... |   | $M$ |   | ... |   |

- \* chercher le cours maximal ( $M'$ ) postérieur au cours minimal  $m$

|       |   |   |     |   |   |     |     |      |
|-------|---|---|-----|---|---|-----|-----|------|
| Jours | 0 | 1 | 2   | 3 | 4 | 5   | 6   | 7    |
| Cours |   |   | ... |   |   | $m$ | ... | $M'$ |

- \* renvoyer la meilleure paire ( $M, m'$ ) ou ( $M', m$ )

- \* Efficace ?

## » Une deuxième proposition d'algorithme

Soit acheter au prix minimal soit vendre au jour maximal

- \* idée de l'algorithme :

- \* trouver le cours maximal ( $M$ ) et minimal ( $m$ )

- \* chercher le cours minimal ( $m'$ ) antérieur au cours maximal  $M$

|       |   |      |     |   |     |   |     |   |
|-------|---|------|-----|---|-----|---|-----|---|
| Jours | 0 | 1    | 2   | 3 | 4   | 5 | 6   | 7 |
| Cours |   | $m'$ | ... |   | $M$ |   | ... |   |

- \* chercher le cours maximal ( $M'$ ) postérieur au cours minimal  $m$

|       |   |   |     |   |   |     |     |      |
|-------|---|---|-----|---|---|-----|-----|------|
| Jours | 0 | 1 | 2   | 3 | 4 | 5   | 6   | 7    |
| Cours |   |   | ... |   |   | $m$ | ... | $M'$ |

- \* renvoyer la meilleure paire ( $M, m'$ ) ou ( $M', m$ )

- \* Efficace ? Correct ?

## » Une deuxième proposition d'algorithme

Pas correct...

| Jours | 0   | 1          | 2   | 3  | 4   | 5   | 6         | 7  |
|-------|-----|------------|-----|----|-----|-----|-----------|----|
| Cours | 100 | <u>113</u> | 110 | 85 | 102 | 105 | <u>63</u> | 70 |

$$* (100, 113) \Rightarrow +13$$

$$* (63, 70) \Rightarrow +7$$

## » Une deuxième proposition d'algorithme

Pas correct...

| Jours | 0   | 1          | 2   | 3  | 4   | 5   | 6         | 7  |
|-------|-----|------------|-----|----|-----|-----|-----------|----|
| Cours | 100 | <u>113</u> | 110 | 85 | 102 | 105 | <u>63</u> | 70 |

$$* (100, 113) \Rightarrow +13$$

$$* (63, 70) \Rightarrow +7$$

$$* (85, 105) \Rightarrow +20$$

## » Diviser pour régner

- \* On transforme le tableau en écarts (= différences de valeurs) quotidiens :

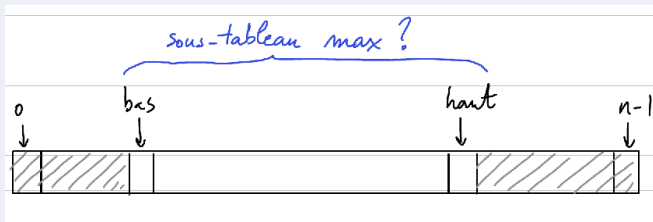
| Jours  | 0   | 1   | 2   | 3   | 4   | 5   | 6   | 7  |
|--------|-----|-----|-----|-----|-----|-----|-----|----|
| Cours  | 100 | 113 | 110 | 85  | 102 | 105 | 63  | 70 |
| écarts |     | 13  | -3  | -25 | 17  | 3   | -42 | 7  |

- \* Trouver les jours d'achat et de vente optimaux est équivalent à trouver le sous-tableau dont la somme des écarts est maximale.



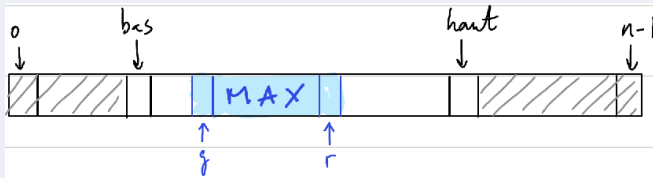
## » Diviser pour régner

On cherche à écrire l'algorithme `SOUS-TABLEAU-MAXIMAL (A, bas, haut)` qui renvoie le sous-tableau maximal de la partie du tableau `A` délimitée par les indices `haut` et `bas` (c'est-à-dire  $A[bas..haut]$ )



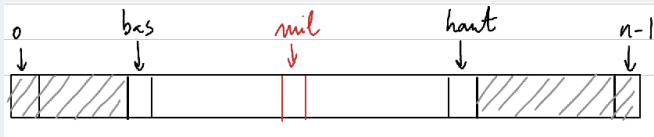
## » Diviser pour régner

C'est-à-dire un sous-tableau  $A[g..r]$  avec  $bas \leq g \leq r \leq haut$



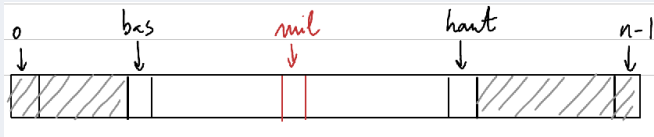
## » Diviser pour régner

Diviser : on coupe  $A[bas \dots haut]$  en deux :  $A[bas..mil]$  et  $A[mil + 1..haut]$



## » Diviser pour régner

Diviser : on coupe  $A[bas \dots haut]$  en deux :  $A[bas..mil]$  et  $A[mil + 1..haut]$



et trois cas de figure se présentent pour le sous-tableau maximal  $A[g..r]$  vis-à-vis de ce découpage !

## » Diviser pour régner

Soit  $A[g..r] \subseteq A[bas..mil]$  :



## » Diviser pour régner

Soit  $A[g..r] \subseteq A[bas..mil]$  :



Soit  $A[g..r] \subseteq A[mil + 1..haut]$  :



## » Diviser pour régner

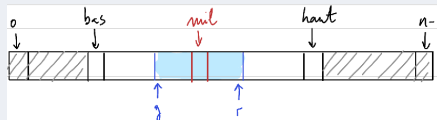
Soit  $A[g..r] \subseteq A[bas..mil]$  :



Soit  $A[g..r] \subseteq A[mil + 1..haut]$  :



Soit  $g \leq mil$  et  $r \geq mil + 1$  :



## » Sous-tableau maximal du milieu

SOUS-TABLEAU-MILIEU(A,bas,mil,haut)

sommeGauche =  $-\infty$

somme = 0

**for** i=mil downto bas

    somme = somme + A[i]

**if** somme > sommeGauche then

        sommeGauche = somme

        maxGauche = i

sommeDroite =  $-\infty$

somme = 0

**for** j=mil+1 to haut

    somme = somme + A[j]

**if** somme > sommeDroite then

        sommeDroite = somme

        maxDroite = j

**return** (maxGauche,maxDroite,sommeGauche+sommeDroite)

Coût de TROUVER-SOUS-TABLEAU-MILIEU(A,bas,mil,haut) ?



## » Sous-tableau maximal du milieu

SOUS-TABLEAU-MILIEU(A,bas,mil,haut)

sommeGauche =  $-\infty$

somme = 0

**for** i=mil downto bas

    somme = somme + A[i]

**if** somme > sommeGauche then

        sommeGauche = somme

        maxGauche = i

sommeDroite =  $-\infty$

somme = 0

**for** j=mil+1 to haut

    somme = somme + A[j]

**if** somme > sommeDroite then

        sommeDroite = somme

        maxDroite = j

**return** (maxGauche,maxDroite,sommeGauche+sommeDroite)

Coût de TROUVER-SOUS-TABLEAU-MILIEU(A,bas,mil,haut) ?

$$\Theta(n) \text{ où } n = \text{haut} - \text{bas} + 1$$

## » Sous-tableau maximal

SOUS-TABLEAU-MAXIMAL(A,bas,haut)

```
if haut==bas then
    return (bas,haut,A[bas])
else
    mil = inf((bas+haut)/2)
    (basGauche,hautGauche,sommeGauche) = SOUS-TABLEAU-MAXIMAL(A,bas,mil)
    (basDroite,hautDroite,sommeDroite) = SOUS-TABLEAU-MAXIMAL(A,mil+1,haut)
    (basMil,hautMil,sommeMil) = SOUS-TABLEAU-MILIEU(A,bas,mil,haut)
    if sommeGauche>sommeDroite and sommeGauche>sommeMil then
        return (basGauche,hautGauche,sommeGauche)
    else if sommeDroite>sommeGauche and sommeDroite>sommeMil then
        return (basDroite,hautDroite,sommeDroite)
    else return (basMil,hautMil,sommeMil)
```

Coût  $T(n)$  de cet algorithme ?

## » Sous-tableau maximal

SOUS-TABLEAU-MAXIMAL(A,bas,haut)

```
if haut==bas then
    return (bas,haut,A[bas])
else
    mil = inf((bas+haut)/2)
    (basGauche,hautGauche,sommeGauche) = SOUS-TABLEAU-MAXIMAL(A,bas,mil)
    (basDroite,hautDroite,sommeDroite) = SOUS-TABLEAU-MAXIMAL(A,mil+1,haut)
    (basMil,hautMil,sommeMil) = SOUS-TABLEAU-MILIEU(A,bas,mil,haut)
    if sommeGauche>sommeDroite and sommeGauche>sommeMil then
        return (basGauche,hautGauche,sommeGauche)
    else if sommeDroite>sommeGauche and sommeDroite>sommeMil then
        return (basDroite,hautDroite,sommeDroite)
    else return (basMil,hautMil,sommeMil)
```

Coût  $T(n)$  de cet algorithme ?

$$T(n) = \begin{cases} \Theta(1) & \text{si } n = 1; \\ 2T(n/2) + \Theta(n) & \text{si } n > 1 \end{cases}$$

## » Sous-tableau maximal

SOUS-TABLEAU-MAXIMAL(A,bas,haut)

```
if haut==bas then
    return (bas,haut,A[bas])
else
    mil = inf((bas+haut)/2)
    (basGauche,hautGauche,sommeGauche) = SOUS-TABLEAU-MAXIMAL(A,bas,mil)
    (basDroite,hautDroite,sommeDroite) = SOUS-TABLEAU-MAXIMAL(A,mil+1,haut)
    (basMil,hautMil,sommeMil) = SOUS-TABLEAU-MILIEU(A,bas,mil,haut)
    if sommeGauche>sommeDroite and sommeGauche>sommeMil then
        return (basGauche,hautGauche,sommeGauche)
    else if sommeDroite>sommeGauche and sommeDroite>sommeMil then
        return (basDroite,hautDroite,sommeDroite)
    else return (basMil,hautMil,sommeMil)
```

Coût  $T(n)$  de cet algorithme ?

$$T(n) = \begin{cases} \Theta(1) & \text{si } n = 1; \\ 2T(n/2) + \Theta(n) & \text{si } n > 1 \end{cases} \Rightarrow T(n) = \Theta(n \log n)$$

## » Tri rapide (Quicksort)

- \* proposé par Tony Hoare en 1961
- \* algorithme « en-place » (i.e. ne nécessite pas d'espace mémoire additionnel)
- \* complexité dans le cas le plus défavorable en  $\Theta(n^2)$
- \* ... mais en moyenne  $\Theta(n \log(n))$  !

## » Tri rapide

- \* suit le paradigme « diviser pour régner » pour trier  $A[p \dots r]$  :
  - \* **Diviser** : partitionnement de  $A[p \dots r]$  en  $A[p \dots q - 1]$  et  $A[q + 1 \dots r]$  tels que chaque élément de  $A[p \dots q - 1]$  est inférieur ou égal à  $A[q]$  (appelé **pivot**) et chaque élément de  $A[q + 1 \dots r]$  est supérieur ou égal à  $A[q]$ .
  - \* **Résolution** du tri (récuratif) de chacune des 2 partitions
  - \* **Combiner** : rien à faire car chaque sous-tableau trié est déjà positionné par rapport au pivot, donc tout le tableau est trié.

## » Tri rapide

- \* suit le paradigme « diviser pour régner » pour trier  $A[p \dots r]$  :
  - \* **Diviser** : partitionnement de  $A[p \dots r]$  en  $A[p \dots q - 1]$  et  $A[q + 1 \dots r]$  tels que chaque élément de  $A[p \dots q - 1]$  est inférieur ou égal à  $A[q]$  (appelé **pivot**) et chaque élément de  $A[q + 1 \dots r]$  est supérieur ou égal à  $A[q]$ .
  - \* **Résolution** du tri (récuratif) de chacune des 2 partitions
  - \* **Combiner** : rien à faire car chaque sous-tableau trié est déjà positionné par rapport au pivot, donc tout le tableau est trié.

```
TRI_RAPIDE(A, p, r)
  if p < r
    q = PARTITION(A, p, r)
    TRI-RAPIDE(A, p, q-1)
    TRI-RAPIDE(A, q+1, r)
```

## » Partitionnement

Partitionnement de  $A[p \dots r]$  : on choisit (par exemple)  $A[r]$  comme pivot et on gère des indices  $i$  et  $j$  tels que :



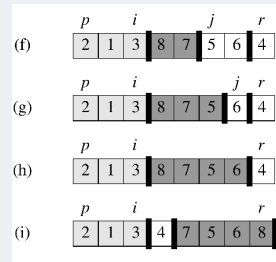
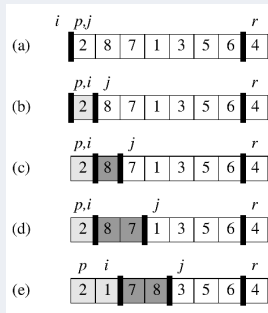


## » Partitionnement

```

PARTITION(A,p,r)
  x = A[r]
  i = p-1
  for j=p to r-1
    if A[j] <= x
      i = i+1
      permuter A[i] <-> A[j]
  permuter A[i+1] <-> A[r]
  return i+1
    
```

### Exemple :



## » Complexité (cas le plus défavorable)

Calcul du coût  $T(n)$  dans le **cas le plus défavorable** :

- \* le partitionnement produit 2 sous-tableaux de taille  $n - 1$  et  $0 \Rightarrow \Theta(n)$ ,
- \* plus un appel récursif sur le sous-tableau de taille  $0 \Rightarrow T(0) = \Theta(1)$ ,
- \* plus un appel récursif sur le sous-tableau de taille  $n - 1 \Rightarrow T(n - 1)$ .

## » Complexité (cas le plus défavorable)

Calcul du coût  $T(n)$  dans le **cas le plus défavorable** :

- \* le partitionnement produit 2 sous-tableaux de taille  $n - 1$  et  $0 \Rightarrow \Theta(n)$ ,
- \* plus un appel récursif sur le sous-tableau de taille  $0 \Rightarrow T(0) = \Theta(1)$ ,
- \* plus un appel récursif sur le sous-tableau de taille  $n - 1 \Rightarrow T(n - 1)$ .

d'où

$$T(n) = \begin{cases} \Theta(1) & \text{si } n = 0 \\ T(n - 1) + \Theta(n) & \text{si } n \geq 1 \end{cases}$$

## » Complexité (cas le plus défavorable)

Calcul du coût  $T(n)$  dans le **cas le plus défavorable** :

- \* le partitionnement produit 2 sous-tableaux de taille  $n - 1$  et  $0 \Rightarrow \Theta(n)$ ,
- \* plus un appel récursif sur le sous-tableau de taille  $0 \Rightarrow T(0) = \Theta(1)$ ,
- \* plus un appel récursif sur le sous-tableau de taille  $n - 1 \Rightarrow T(n - 1)$ .

d'où

$$T(n) = \begin{cases} \Theta(1) & \text{si } n = 0 \\ T(n - 1) + \Theta(n) & \text{si } n \geq 1 \end{cases}$$

qui a pour solution ...

## » Complexité (cas le plus défavorable)

Calcul du coût  $T(n)$  dans le **cas le plus défavorable** :

- \* le partitionnement produit 2 sous-tableaux de taille  $n - 1$  et  $0 \Rightarrow \Theta(n)$ ,
- \* plus un appel récursif sur le sous-tableau de taille  $0 \Rightarrow T(0) = \Theta(1)$ ,
- \* plus un appel récursif sur le sous-tableau de taille  $n - 1 \Rightarrow T(n - 1)$ .

d'où

$$T(n) = \begin{cases} \Theta(1) & \text{si } n = 0 \\ T(n - 1) + \Theta(n) & \text{si } n \geq 1 \end{cases}$$

qui a pour solution ...  $\Theta(n^2)$ .

## » Complexité (cas le plus favorable)

Calcul du coût  $T(n)$  dans le **cas le plus favorable** :

- \* le partitionnement produit 2 sous-tableaux de taille  $n/2 \Rightarrow \Theta(n)$ ,
- \* plus 2 appels récurifs sur des sous-tableaux de taille  $n/2 \Rightarrow T(n/2)$

## » Complexité (cas le plus favorable)

Calcul du coût  $T(n)$  dans le **cas le plus favorable** :

- \* le partitionnement produit 2 sous-tableaux de taille  $n/2 \Rightarrow \Theta(n)$ ,
- \* plus 2 appels récurifs sur des sous-tableaux de taille  $n/2 \Rightarrow T(n/2)$

d'où

$$T(n) = \begin{cases} \Theta(1) & \text{si } n = 0 \\ 2T(n/2) + \Theta(n) & \text{si } n \geq 1 \end{cases}$$

## » Complexité (cas le plus favorable)

Calcul du coût  $T(n)$  dans le **cas le plus favorable** :

- \* le partitionnement produit 2 sous-tableaux de taille  $n/2 \Rightarrow \Theta(n)$ ,
- \* plus 2 appels récurifs sur des sous-tableaux de taille  $n/2 \Rightarrow T(n/2)$

d'où

$$T(n) = \begin{cases} \Theta(1) & \text{si } n = 0 \\ 2T(n/2) + \Theta(n) & \text{si } n \geq 1 \end{cases}$$

qui a pour solution  $\Theta(n \log(n))$ .



## » Complexité (en moyenne)

- \* En moyenne (et presque toujours), le tri rapide a une complexité asymptotique de  $\Theta(n \log(n))$ .

## » Complexité (en moyenne)

- \* En moyenne (et presque toujours), le tri rapide a une complexité asymptotique de  $\Theta(n \log(n))$ .
- \* pire cas en  $\Theta(n^2)$  mais très improbable

## » Complexité (en moyenne)

- \* En moyenne (et presque toujours), le tri rapide a une complexité asymptotique de  $\Theta(n \log(n))$ .
- \* pire cas en  $\Theta(n^2)$  mais très improbable
- \* pour « éliminer » le pire cas : choisir le pivot aléatoirement

## Emacs 30.2 (Org mode 9.7.34)